Лабораторная работа № 18-19

Цикл for и обход элементов foreach в C#**foreach в C#**

Цель работы: Научиться использовать for для диапазонов, известного количества повторений и работы по индексам; foreach - для последовательного чтения готовых элементов. Подготовить полезные заготовки: алфавиты, таблицы, последовательности и обработку наборов данных.foreach — для последовательного чтения готовых элементов. Подготовить полезные заготовки: алфавиты, таблицы, последовательности и обработку наборов данных.

Общие упражнения

1. Сформируйте латинский алфавит приведённой программой. Проверьте длину 26, первую A и последнюю Z.

```
char[] alfanit = new char[26];
for (int i = 0; i < alfanit.Length; i++)
   alfanit[i] = (char)('A' + i);
foreach (char letter in alfanit)
   Console.Write(letter + " ");
Console.WriteLine();
Console.WriteLine($"Длина: {alfanit.Length}");
Console.WriteLine($"Первая: {alfanit[0]}, последняя: {alfanit[25]}");
```

Результат: A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

Длина 26, первая A, последняя Z - проверки пройдены.

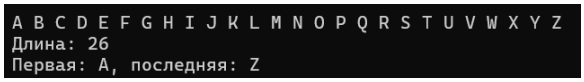


Рисунок 1 - результат работы

2. Создайте русский алфавит из 33 букв с Ё после Е. Проверки: первые А, Б; позиции 6 и 7 при нумерации с 1 - Е и Ё; последняя Я. Можно пройти диапазон А…Я и вставить Ё после Е отдельной записью.

```
char[] Rusalfavit = new char[33];
int index = 0;
for (char c = 'А'; c <= 'Я'; c++)
{
   Rusalfavit[index++] = c;
   if (c == 'Е')
      Rusalfavit[index++] = 'Ё';
}
foreach (char letter in Rusalfavit)
   Console.Write(letter + " ");
Console.WriteLine();
Console.WriteLine($"Позиция 6: {Rusalfavit[5]}, позиция 7: {Rusalfavit[6]}");
Console.WriteLine($"Последняя: {Rusalfavit[32]}");
```

Результат: А Б В Г Д Е Ё Ж З И Й К Л М Н О П Р С Т У Ф Х Ц Ч Ш Щ Ъ Ы Ь Э Ю Я

Длина 33, позиции 6 и 7 - Е и Ё, последняя Я.



Рисунок 2 - результат работы

3. Через for заполните int[5] числами 1…5, через foreach получите сумму 15. Объясните разные роли циклов.

```
int[] num =  new int[5];
long sum = 0;
for (int i = 0; i < num.Length; i++)
{
   num[i] = i + 1;
}
foreach (int znach in num)
   sum += znach;
Console.WriteLine($"Сумма: {sum}");
```



Рисунок 3 - результат работы

4. Через foreach отберите непустые строки готового массива в новый List. Исходник не меняйте.

```
string[] stroka = { "Polina", "Arina", "", "Karina"};
List<string> res = new List<string>();
foreach (string s in stroka)
   if (s.Length > 0)
      res.Add(s);
foreach (string s in res)
   Console.WriteLine ("Отобрано: " + s);
```

Результат: Отобрано: Polina, Arina, Karina. Исходный массив не менялся - foreach только читает элементы, а добавление идёт в другой список.

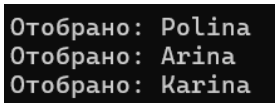


Рисунок 4 - результат работы

5. Проверьте пустой массив и массив из одного элемента. Объясните разницу «чётный индекс» и «чётное значение».

```
int[] empty = { };
int[] one = { 7 };
foreach (int v in empty) Console.Write("Пустой массив: " +  v);
Console.WriteLine("Тело не выполнилось");
foreach (int v in one) Console.Write("Массив с одним элементом: " + v);
```

Чётный индекс - позиция элемента в массиве (0, 2, 4...). Для его проверки нужен цикл for и условие i % 2 == 0.

Чётное значение - это само число, которое делится на 2 без остатка (2, 4, 6...). Для его проверки достаточно foreach и условия v % 2 == 0.

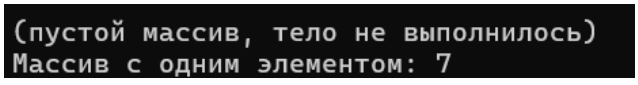


Рисунок 5- результат работы

Вариант 6**Вариант 6**

А1. Через for найдите индексы всех нулей в int[]; индексы начинаются с 0.
Вход: массив int[].
Выход: индексы нулей (с 0).
Ограничения: массив может быть пустым.

```
int[] zero = [-3, 1, 5, 0, 7, 0, 0, 0];
for (int i = 0; i < zero.Length; i++)
{
   if (zero[i] == 0)
      Console.WriteLine(i + "");
}
```

Обьяснение: нужен номер позиции (индекс) - foreach его не даёт.
Тесты:

Вход	Ожидаемый результат	Фактический
{5, 0, -3, 0, 0, 8}	индексы 1 3 4	1 3 4
{}	нулей нет	нулей нет
{0}	индекс 0	0

Трассировка:

Проход	i до	zero[i]	Условие zero[i]==0	Действие	i после
1	0	-3	false	-	1
2	1	1	false	-	2
3	2	5	false	-	3
4	3	0	true	вывод 3	4
5	4	7	false	-	5
6	5	0	true	вывод 5	6
7	6	0	true	вывод 6	7
8	7	0	true	вывод 7	8
-	8	-	8 < 8 ложно	выход	-

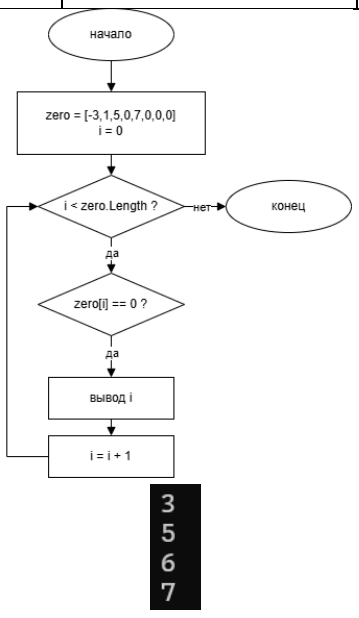


Рисунок 6- результат работы

Б1. По готовому массиву мест (0 - свободно, 1 - занято) выведите номера свободных мест начиная с 1.
Вход: готовый массив мест.

Выход: номера свободных мест (с 1).

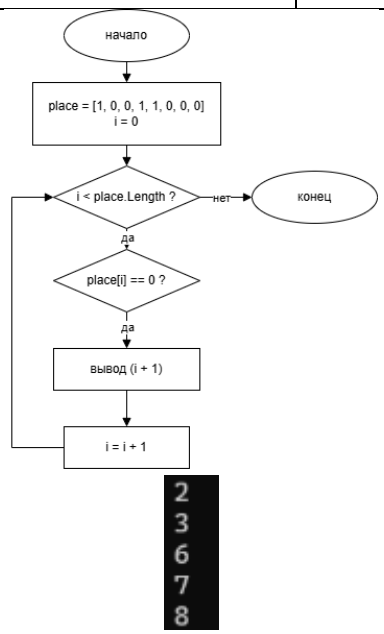
Ограничения: пустой массив допустим.

```
int[] place = [1, 0, 0, 1, 1, 0, 0, 0];
for (int i = 0; i < place.Length; i++)
{
   if (place[i] == 0)
      Console.WriteLine((i + 1) + "");
}
```

Обьяснение: Выбран for, так как нужен индекс i для вычисления номера места (i + 1). foreach не даёт доступа к позиции.

Тесты:

Вход	Ожидаемый	Фактический
{1,0,0,1,0}	2 3 5	2 3 5
{}	свободных мест нет	свободных мест нет
{1,1}	свободных мест нет	свободных мест нет




```
2
3
6
7
8
```

Рисунок 7- результат работы

А2. Через foreach подсчитайте пустые строки string[]; элементы не null.

Вход: string[] (элементы не null).

Выход: число пустых строк.

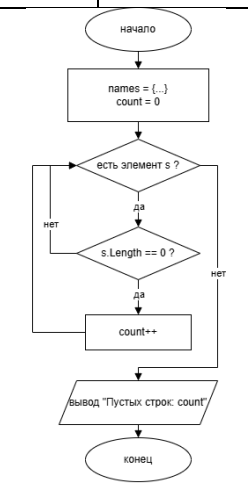
Ограничения: пустой массив → 0.

```
string[] names = { "Анна", "", "Иван", "", "" };
int count = 0;
foreach (string s in names)
{
   if (s.Length == 0)
      count++;
}
Console.WriteLine($"Пустых строк: {count}");
```

Обьяснение: foreach, так как индекс не нужен, важны только сами значения строк для проверки их длины.

Тесты:

Вход	Ожидаемый	Фактический
{"Анна","","Иван","",""}	3	3
{}	0	0
{"",""}	2	2




```
Пустых строк: 3
```

Рисунок 8- результат работы

Б2. По готовому списку имён определите число записей, пустых после Trim. null в данных отсутствует.

Вход: готовый список имён (null отсутствует).

Выход: число записей, пустых после Trim().

Ограничения: пустой список → 0.

```
string[] names = { "Анна", "  ", "\t", "Иван", "  " };
int empty = 0;
foreach (string s in names)
{
   if (s.Trim().Length == 0)
      empty++;
}
Console.WriteLine($"Пустых после Trim: {empty}");
```

Обьяснение: foreach, так как позиция элемента не важна, обрабатываются только значения.

Тесты:

Вход	Ожидаемый	Фактический
{"Анна"," ","\t","Иван"," "}	3	3
{}	0	0
{" ","\t\t"}	2	2

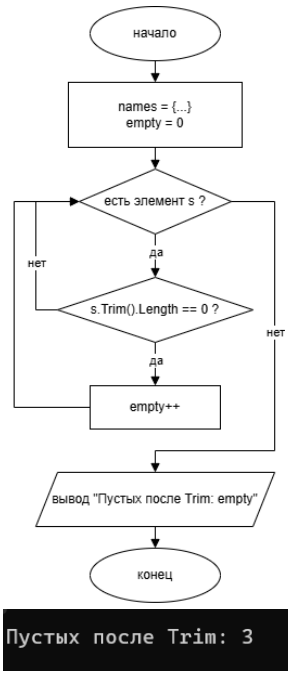


Рисунок 9 - результат работы

Контрольные вопросы

1. В каком порядке выполняются части for?

Ответ: Инициализация (1 раз) → проверка условия → тело → изменение счётчика → снова проверка.

2. Почему условие i < Length корректно для массива?

Ответ: Индексы начинаются с 0, последний - Length - 1. Условие < не даёт выйти за границы.

3. Как организовать обратный обход пустого массива?

Ответ: Старт с Length - 1 (будет -1), условие i >= 0. Цикл не выполнится, ошибки нет.

4. Когда foreach удобнее for?

Ответ: Когда индекс не нужен, важны только значения (сумма, подсчёт, поиск).

5. Почему foreach не используют для замены элементов массива?

Ответ: Переменная цикла - копия значения, а не ссылка. Для замены нужен for и arr[i] = ...

6. Можно ли изменять тот же List при foreach?

Ответ: Нет, при Add или Remove будет ошибка InvalidOperationException.

7. Почему Ё нужно добавлять отдельно?

Ответ: Она не входит в непрерывный диапазон кодов от А до Я, её вставляют вручную после Е.

8. Чем чётные индексы отличаются от чётных значений?

Ответ: Индекс - позиция (i % 2 == 0), значение - само число (value % 2 == 0).

9. Как проверить отсутствие подходящих элементов?
Ответ: До цикла задать флаг found = false или count = 0, проверить их после цикла.
10. Сколько действий даёт вложенный цикл R × C?
Ответ: Ровно R * C действий (внешний умножить на внутренний).
Вывод: я научилась применять циклы for и foreach для обработки наборов данных. Убедилась, что при обходе пустого массива циклы не вызывают ошибок - тело цикла просто не выполняется ни разу, что обеспечивает безопасность программы.